

الدرس 6 – بطاقة مرجعية سريعة

Model إنشاء ؟

```
# Model بسيط
php artisan make:model Product

# مع Migration
php artisan make:model Product -m

# كل شيء
php artisan make:model Product -mfsc
# -m: migration, -f: factory, -s: seeder, -c: controller
```

استرجاع البيانات ؟

```
// الكل
$products = Product::all();
$products = Product::all(['name', 'price']);

// بالمفتاح
$product = Product::find(1);
$products = Product::find([1, 2, 3]);
$product = Product::findOrFail(1);

// Where
$products = Product::where('price', '>', 100)->get();
$products = Product::where('active', 1)->get();
$products = Product::whereIn('id', [1, 2, 3])->get();
$products = Product::whereBetween('price', [100, 500])->get();

// أول سجل
$product = Product::first();
$product = Product::where('slug', 'laptop')->first();
$product = Product::firstOrFail();

// firstOrCreate
$product = Product::firstOrCreate(
    ['slug' => 'laptop'],
    ['name' => 'Laptop', 'price' => 5000]
```

```
);

// الترتيب
$products = Product::orderBy('price', 'desc')->get();
$products = Product::latest()->get();
$products = Product::oldest()->get();

// التحديد
$products = Product::take(10)->get();
$products = Product::skip(10)->take(5)->get();

// العد
$count = Product::count();
$max = Product::max('price');
$min = Product::min('price');
$sum = Product::sum('price');
$avg = Product::avg('price');
```

❑ إنشاء السجلات

```
// create()
$product = Product::create([
    'name' => 'Laptop',
    'price' => 5000,
]);

// save()
$product = new Product;
$product->name = 'Laptop';
$product->price = 5000;
$product->save();

// insert (عدة سجلات)
Product::insert([
    ['name' => 'Product 1', 'price' => 100],
    ['name' => 'Product 2', 'price' => 200],
]);

// updateOrCreate
$product = Product::updateOrCreate(
    ['slug' => 'laptop'],
    ['name' => 'Gaming Laptop', 'price' => 7000]
```

```
);
```

تحديث السجلات ؟؟

```
// save()
$product = Product::find(1);
$product->price = 6000;
$product->save();

// update()
$product = Product::find(1);
$product->update(['price' => 6000]);

// تحديث عدة سجلات
Product::where('category', 'Electronics')
    ->update(['discount' => 10]);

// increment / decrement
$product->increment('views');
$product->increment('views', 5);
$product->decrement('stock');
$product->decrement('stock', 2);
```

حذف السجلات ؟

```
// delete()
$product = Product::find(1);
$product->delete();

// destroy()
Product::destroy(1);
Product::destroy([1, 2, 3]);

// حذف بشرط
Product::where('active', 0)->delete();

// حذف الكل
```

```
Product::truncate();
```

❑ Soft Deletes

```
// في Migration
$table->softDeletes();

// في Model
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use SoftDeletes;
}

// الاستخدام
$product->delete(); // حذف ناعم

// مع المحذوفة
$products = Product::withTrashed()->get();

// المحذوفة فقط
$products = Product::onlyTrashed()->get();

// الاستعادة
$product = Product::withTrashed()->find(1);
$product->restore();

// حذف نهائي
$product->forceDelete();

// التحقق
if ($product->trashed()) {
    // محذوف
}
```

□ Query Scopes

```
// Local Scope في Model
public function scopeActive($query)
{
    return $query->where('active', 1);
}

public function scopeCheap($query, $maxPrice = 100)
{
    return $query->where('price', '<=', $maxPrice);
}

// الاستخدام
$products = Product::active()->get();
$products = Product::active()->cheap(50)->get();

// Global Scope
protected static function booted()
{
    static::addGlobalScope('active', function ($query) {
        $query->where('active', 1);
    });
}

// تجاهل Global Scope
$products = Product::withoutGlobalScope('active')->get();
```

□ Accessors & Mutators

```
use Illuminate\Database\Eloquent\Casts\Attribute;

// Accessor (القراءة)
protected function name(): Attribute
{
    return Attribute::make(
        get: fn ($value) => ucfirst($value),
    );
}

// Mutator (الكتابة)
protected function name(): Attribute
```

```

{
  return Attribute::make(
    set: fn ($value) => strtolower($value),
  );
}

// معاً
protected function price(): Attribute
{
  return Attribute::make(
    get: fn ($value) => number_format($value, 2) . ' ريال',
    set: fn ($value) => (float) str_replace(',', '', $value),
  );
}

// استخدام
$product = Product::find(1);
echo $product->name; // يمر عبر Accessor
$product->name = 'LAPTOP'; // يمر عبر Mutator

```

□ Mass Assignment

```

// في Model
class Product extends Model
{
  // السماح
  protected $fillable = [
    'name',
    'price',
    'stock',
  ];

  // أو المنع
  protected $guarded = [
    'id',
    'created_at',
    'updated_at',
  ];

  // (السماح بالكل (غير آمن
  protected $guarded = [];

```

```
}
```

Model خصائص

```
class Product extends Model
{
    // اسم الجدول
    protected $table = 'products';

    // المفتاح الأساسي
    protected $primaryKey = 'id';
    protected $keyType = 'int';
    public $incrementing = true;

    // Timestamps
    public $timestamps = true;
    const CREATED_AT = 'created_at';
    const UPDATED_AT = 'updated_at';

    // Fillable / Guarded
    protected $fillable = ['name', 'price'];
    protected $guarded = ['id'];

    // Casts
    protected $casts = [
        'price' => 'float',
        'is_active' => 'boolean',
        'published_at' => 'datetime',
    ];
}
```

Timestamps

```
// تفعيل/تعطيل
public $timestamps = true; // افتراضي
public $timestamps = false; // تعطيل

// تخصيص
```

```
const CREATED_AT = 'creation_date';
const UPDATED_AT = 'last_update';

// حفظ بدون تحديث
$product->saveQuietly();
$product->timestamps = false;
$product->save();

// تحديث Timestamps
$product->touch();
```

❑ Chunking

```
// للبيانات الكبيرة
Product::chunk(100, function ($products) {
    foreach ($products as $product) {
        // معالجة
    }
});

// Lazy Loading
Product::lazy()->each(function ($product) {
    // معالجة
});
```

❑ أمثلة سريعة ؟

```
// إنشاء منتج
$product = Product::create([
    'name' => 'Laptop',
    'price' => 5000,
    'stock' => 10,
]);

// عرض منتجات نشطة ورخيصة
$products = Product::where('active', 1)
    ->where('price', '<', 1000)
    ->orderBy('name')
```



```
->get();

// تحديث المخزون
$product = Product::find(1);
$product->decrement('stock');
$product->increment('sales');

// حذف المنتجات القديمة
Product::where('stock', 0)
    ->where('updated_at', '<', now()->subMonths(6))
    ->delete();

// منتجات شائعة
$popular = Product::where('views', '>', 1000)
    ->orderBy('views', 'desc')
    ->take(10)
    ->get();
```

أفضل الممارسات ؟

أو `$fillable` لإعادة الاستخدام ؟ حدد `Query Scopes` الخام ؟ استخدم `SQL` بدلاً من `Eloquent` استخدم ؟
للبيانات الكبيرة `chunk()` للحذف الآمن ؟ استخدم `Soft Deletes` دائماً ؟ استخدم `$guarded`

في الإنتاج `[$guarded = []]` لا تستخدم `create()` عند استخدام `fillable` للبيانات الكبيرة ؟ لا تنسى `all()` لا تستخدم ؟

روابط سريعة ؟

- الدرس الرئيسي
- الدرس السابق
- الدرس التالي